

如何设计实现微前端框架

QIANKUN

前端早早聊 / 2020.5.30

自我介绍



方涣

@蚂蚁金服 · 体验技术部 · 前端工程师

qiankun 简介

qiankun.umijs.org

qiankun

可能是你见过最完善的微前端解决方案 🤖

快速开始 →

简单

任意 js 框架均可使用。微应用接入像使用接入一个 iframe 系统一样简单，但实际不是 iframe。

完备

几乎包含所有构建微前端系统时所需要的基本能力，如 样式隔离、js 沙箱、预加载等。

生产可用

已在蚂蚁内外经受过足够大量的线上系统的考验及打磨，健壮性值得信赖。

蚂蚁金服 | Qiankun

Qiankun 是一个微前端框架，它可以帮助你快速构建微前端系统。

蚂蚁金服

Qiankun 是一个微前端框架，它可以帮助你快速构建微前端系统。

Qiankun 是一个微前端框架，它可以帮助你快速构建微前端系统。

qiankun 名字的由来

小世界有大乾坤

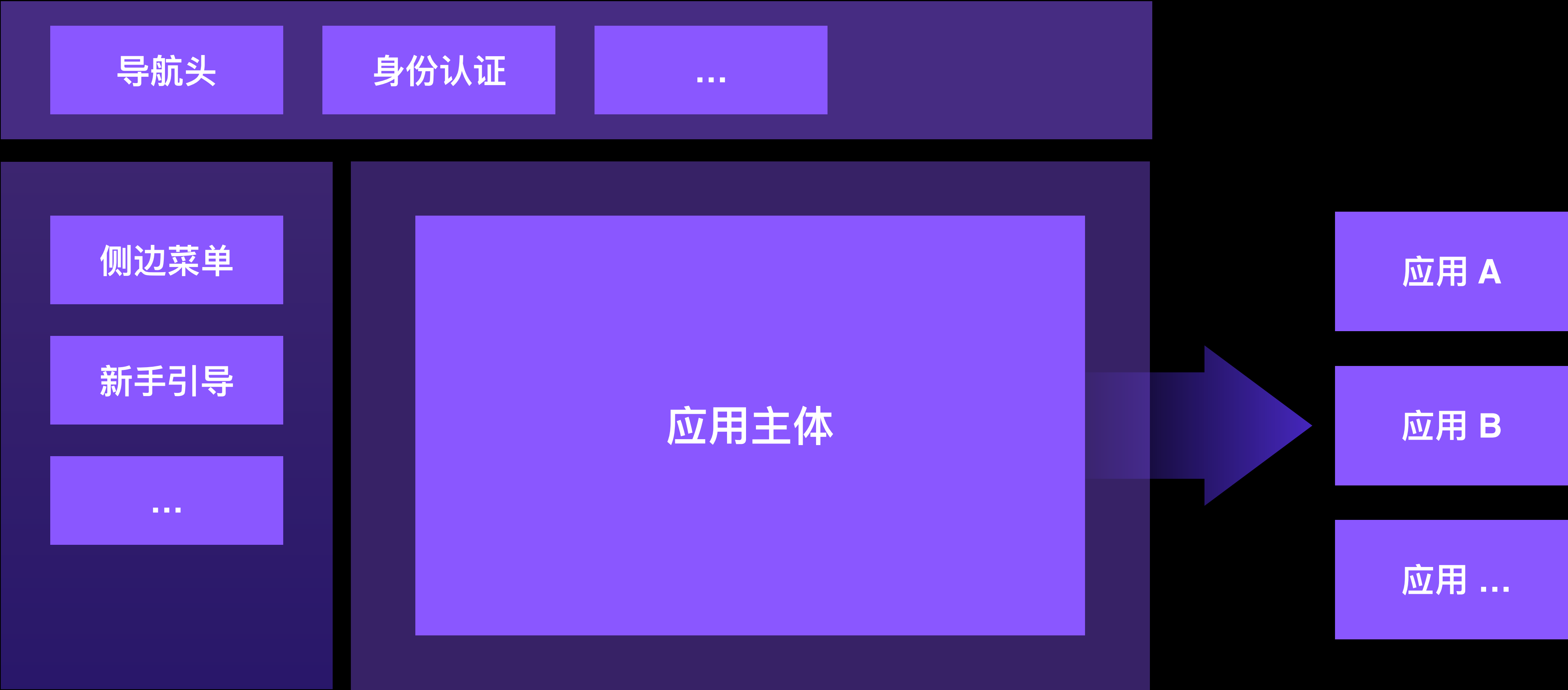
QIANKUN 的诞生与设计理念

第一小节

缘起：一次统一上云战役



控制台结构



我们需要一个“友好”的微前端方案

技术栈无关

不限制接入应用的技术栈
应用之间没有隐性依赖

接入简单

最好就像 iframe 一样简单
尽可能避免旧应用改造

技术栈无关

“技术栈无关”是微前端的核心价值

“技术栈无关”为什么重要？

统一控制台 ABC		
项目 A	项目 B	项目 C
一个月前，积极迭代	一年前，缺乏维护	三年前，无人维护
react@16 @umijs/hooks webpack@5 css modules	react@15 dva webpack@3 css-in-js	jQuery Gulp Sass

微前端的前提，还是得有主体应用，然后才有微组件或微应用，解决的是**可控体系下的前端协同开发问题**

(含 空间分离带来的协作 和 时间延续带来的升级维护)

@玉伯 · 蚂蚁金服

“技术栈无关”的价值

让天下没有短命的控制台

不限制技术栈，接入范围广

向后兼容：兼容年久的旧应用

向前兼容：架构稳定，面向未来

QIANKUN 的技术实现与选择

第二小节

微前端框架面临的两大共性问题

应用的加载与切换

路由

应用加载

应用入口

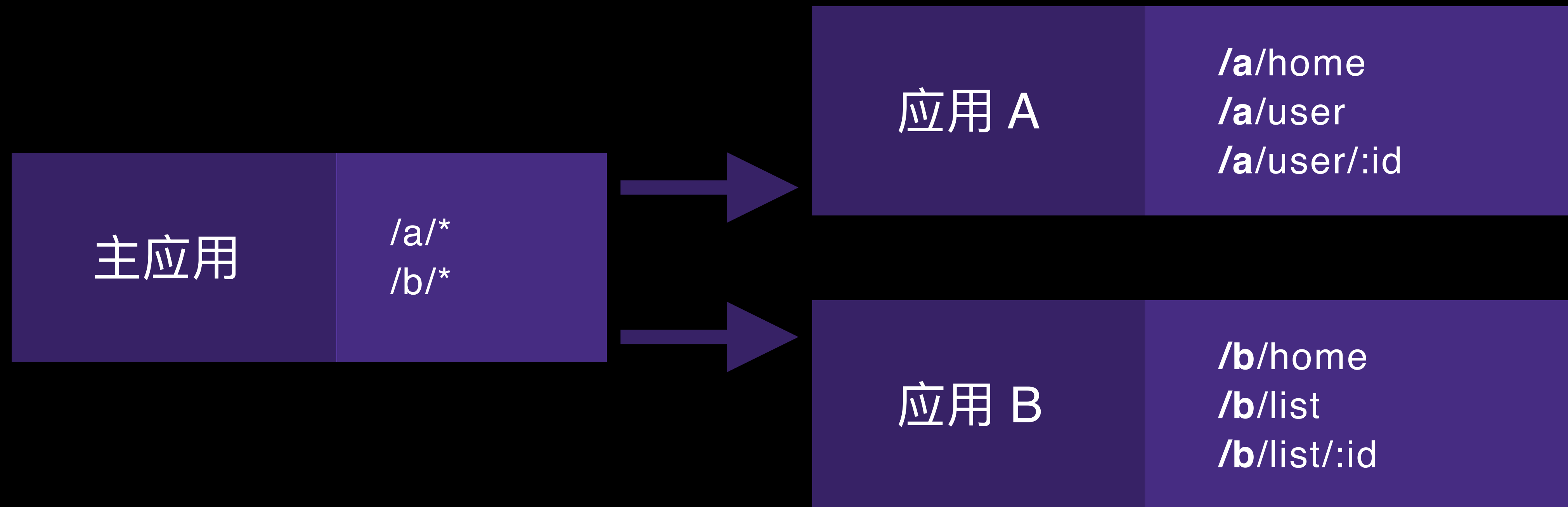
应用的隔离与通信

JS 隔离

样式隔离

父子、子子应用通信

应用路由 与 Future State



如果我们在路由 `/b/list/1` 下刷新页面会发生什么？

社区已有的成熟方案



Single-SPA

应用接入：协议接入



```
export function bootstrap() {  
  // one time initialization  
  return Promise.resolve()  
},  
  
export function mount() {  
  // use a framework to create dom nodes and mount the parcel  
  return Promise.resolve()  
},  
  
export function unmount() {  
  // use a framework to unmount dom nodes and perform other cleanup  
  return Promise.resolve()  
}
```

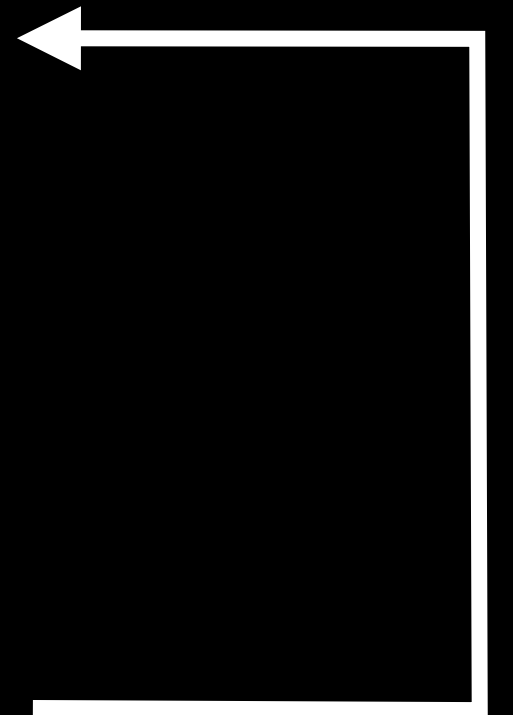
bootstrap



mount



unmount



App Entry 抉择

组合时机

构建时？

运行时？

应用入口

JS？

HTML？

组合时机选择

	说明	优点	缺点
构建时	子应用与主应用一起打包发布	构建时容易做公共依赖提取等优化	主子应用构建方案、工具耦合 主子应用必须一起发布
运行时	子应用独立构建 运行时动态加载	主子应用完全解耦 技术栈无关	有运行时损耗

应用入口选择：HTML Entry



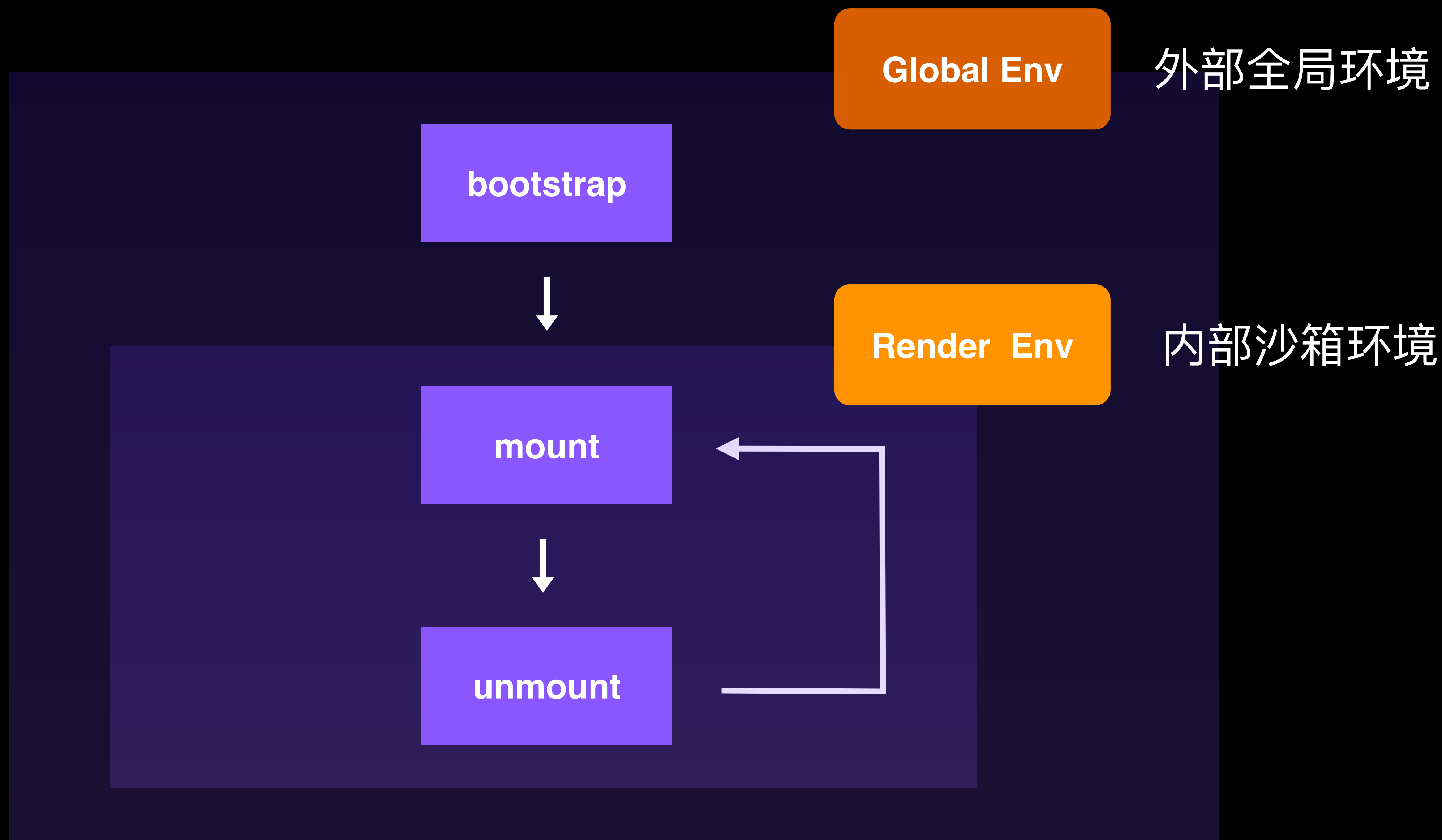
```
<html lang="zh-CN">
  <head>
    <meta charset='utf-8'>
    <meta name="description" content="MicroApp powered by qiankun">
    <!-- CSS -->
    <link rel="stylesheet" href="./style.css" />
  </head>
  <body>
    <!-- dom for mount -->
    <main id="root"></main>
    <!-- js -->
    <script src="./app.js"></script>
  </body>
</html>
```

Entry HTML 中包含了所需的全部信息：网页结构与元信息、js 与 css 等资源

应用入口选择

	说明	优点	缺点
html	html 作为子应用入口	解耦更彻底，子应用不依赖主应用 DOM 子应用独立开发，能独立部署	多了一次对 html 的请求 解析有性能损耗 无法做构建时优化
js	js 作为子应用入口	便于做构建时优化	依赖主应用提供挂载节点 打包产物体积膨胀，资源无法并行加载

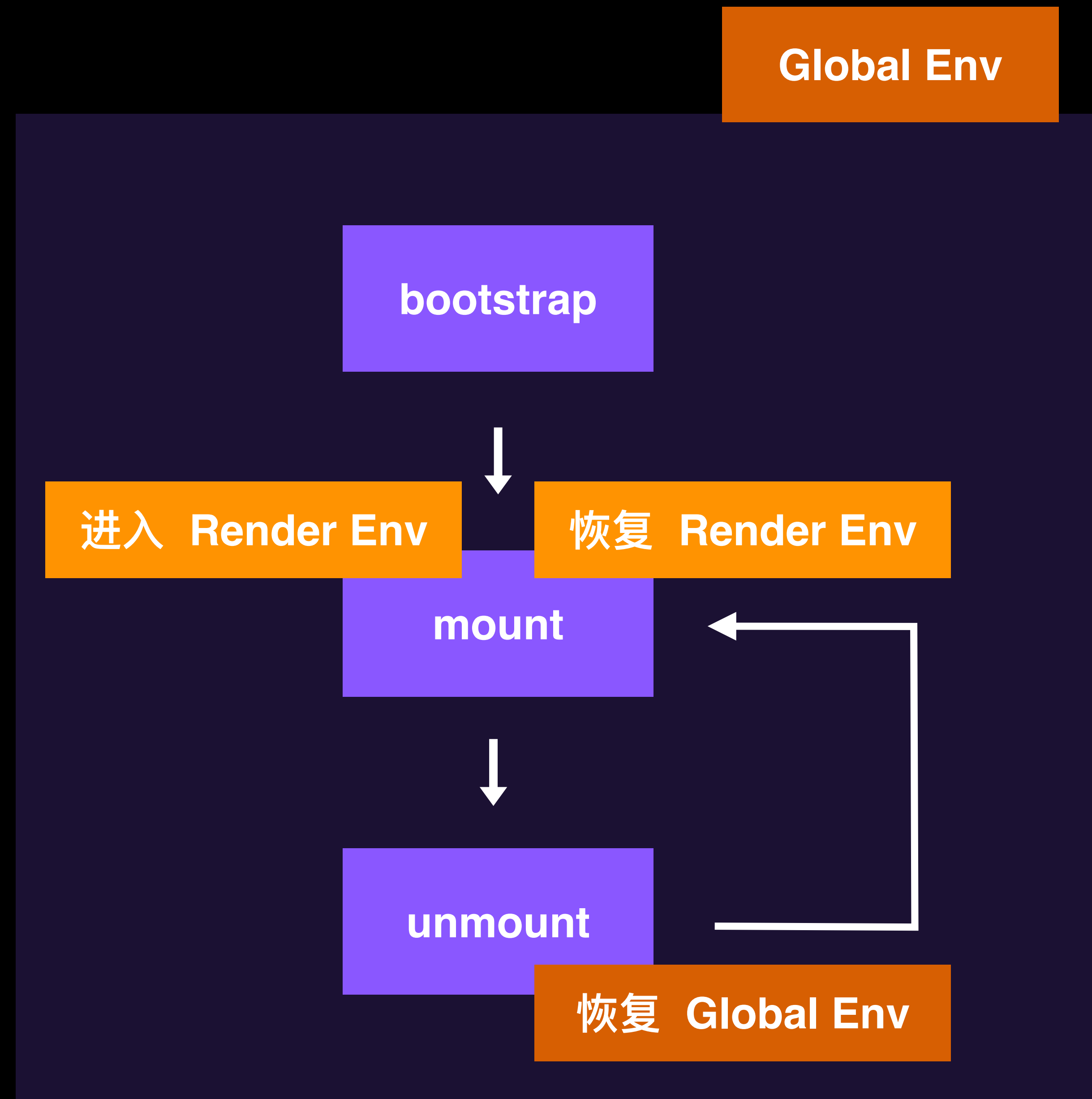
应用隔离：JS 沙箱



应用隔离：JS 沙箱

Snapshot

- 在应用沙箱挂载/卸载时记录快照
- 在应用切换时依据快照恢复环境
- 无法支持多实例



应用隔离：JS 沙箱

Proxy

- 代理沙箱内部操作，不影响 Global Env
- 将副作用局限在 Render Env 内部

Global Env

Env A

应用 A

Env B

应用 B

应用隔离： 样式隔离

```
<style>  
  /* 主应用样式 */  
</style>
```

```
<style>  
  /* 子应用样式 */  
</style>
```

主、子应用样式冲突

```
<style>  
  /* 子应用 A 样式 */  
</style>
```

```
<style>  
  /* 子应用 B 样式 */  
</style>
```

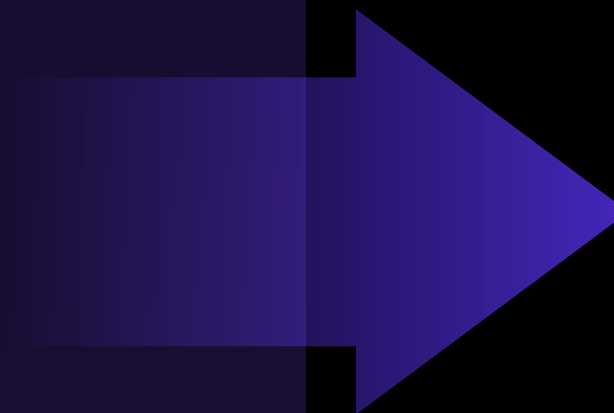
子应用之间样式冲突

样式隔离： Dynamic Stylesheet

```
<style>  
  /* 主应用样式 */  
</style>
```

```
<style>  
  /* 子应用 A 样式 */  
</style>
```

子应用 A 激活



切换到

```
<style>  
  /* 主应用样式 */  
</style>
```

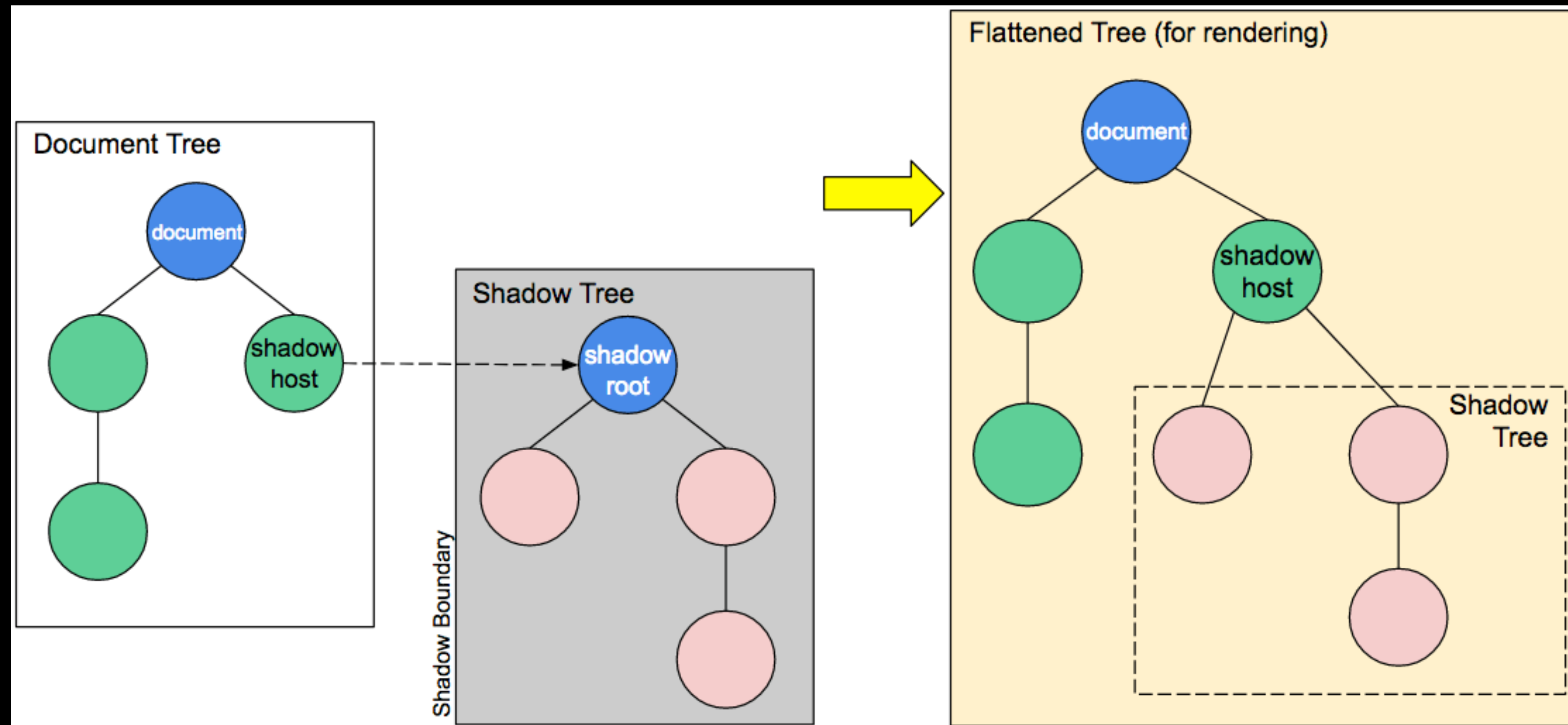
```
<style>  
  /* 子应用 B 样式 */  
</style>
```

子应用 B 激活

样式隔离：工程化手段

	说明	优点	缺点
BEM	不同项目用不同的前缀/命名规则避开冲突	简单	依赖约定，这也是耦合的一种 容易出纰漏
CSS Modules	通过编译生成不冲突的选择器名	可靠易用，避免人工约束	只能在构建期使用 依赖预处理器与打包工具
CSS-in-JS	CSS 和 JS 编码在一起 最终生成不冲突的选择器	基本彻底避免冲突	运行时开销 略缺失完整 css 能力

样式隔离方案：Shadow DOM



样式隔离 RFC: runtime css transformer

```
.main { ... }
```



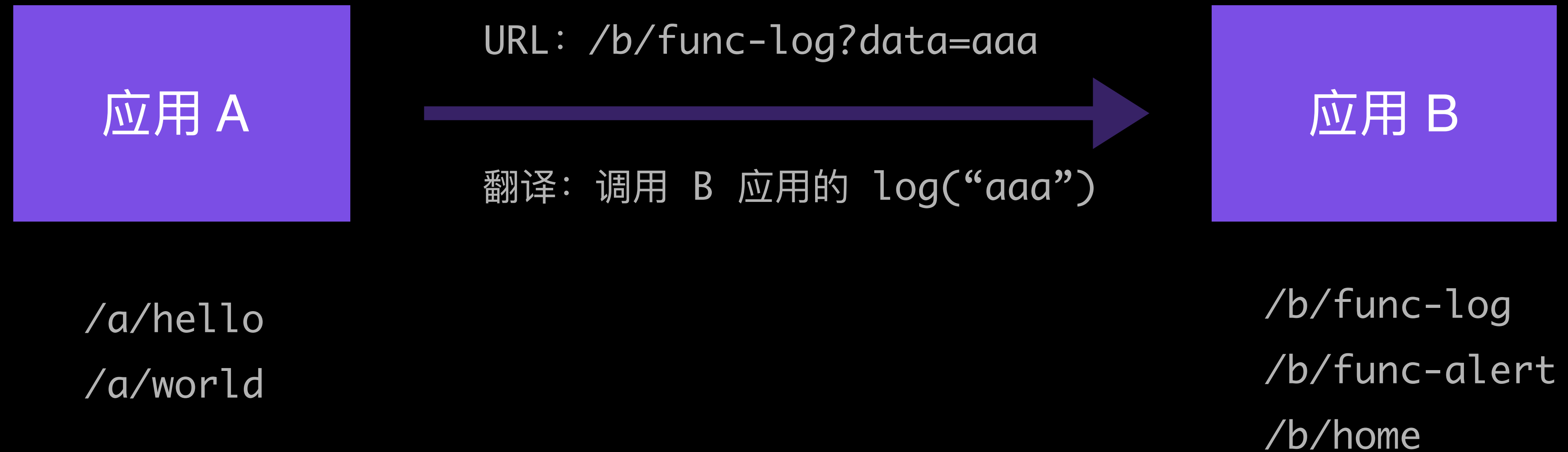
```
div[data-qiankun-app1] .main { ... }
```



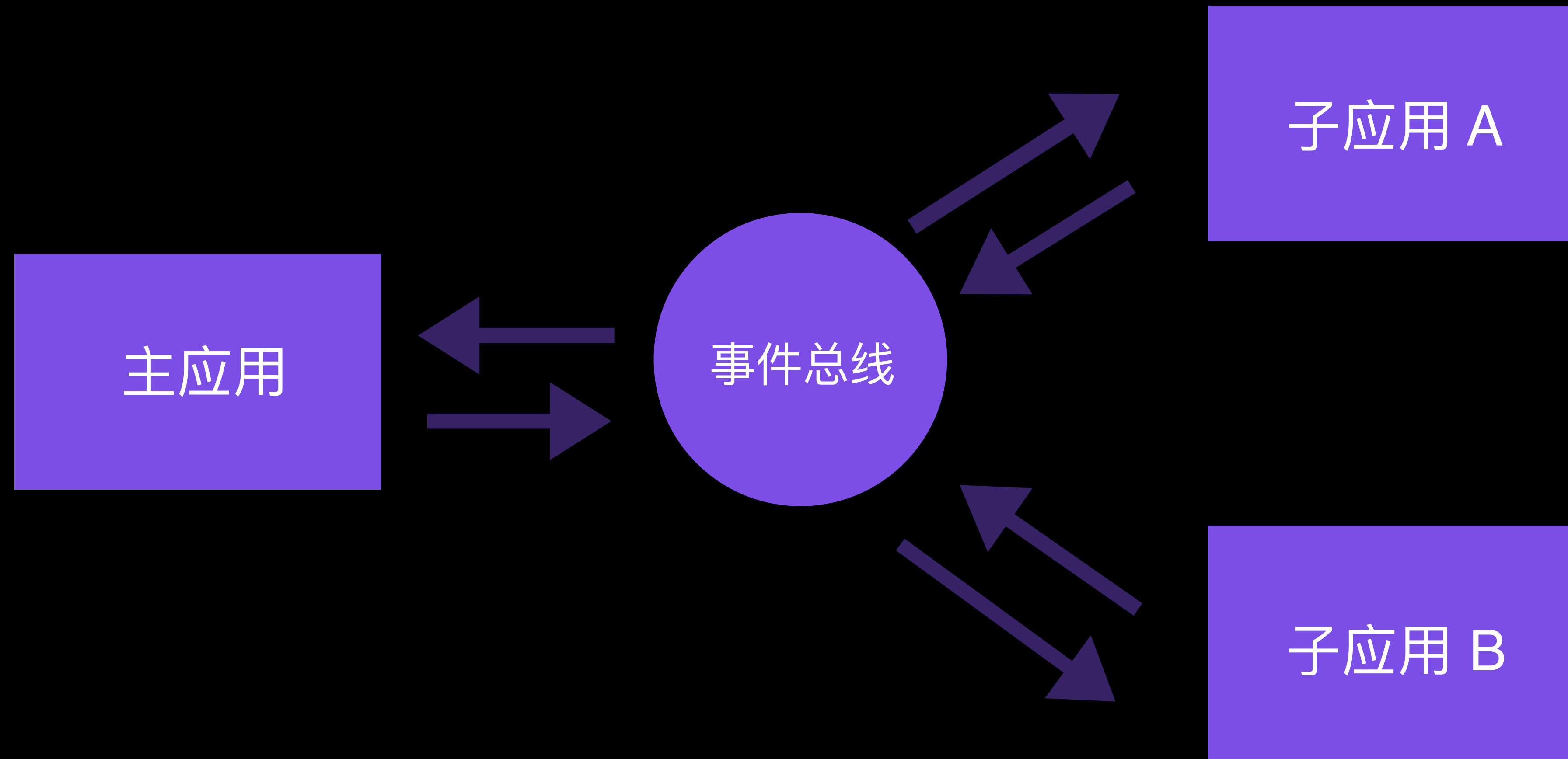
```
div[data-qiankun-应用名] 子应用 css 选择器
```

更多信息可见 qiankun PR#465: <https://github.com/umijs/qiankun/pull/465>

应用通信：基于 URL



应用通信：发布/订阅模型



CustomEvent: 浏览器自带的事件总线

应用通信：基于 props



朴素的主子应用通信

应用通信

	优点	缺点
基于 URL	完全解耦，应用之间独立	传递消息能力弱 符合 URL 中心设计的应用少
基于 CustomEvent	弱耦合 浏览器原生支持	全局命名冲突，缺乏管控 复杂情况下通信零碎、散乱
基于 props	通信能力完全自定义	主子应用耦合较强 新 API 的学习使用成本
全局变量、 共用 Redux	-	强耦合

qiankun 没有解答的工程与平台问题

版本管理

质量管控

配置下发

线上监控

应用编排

公共依赖

灰度发布

安全检测

.....

QIANKUN 的发展与未来

第三小节

qiankun 历程回顾

2019.6

qiankun 正式发布 1.0 版本

官宣文章 [《可能是你见过最完善的微前端解决方案》](#)

2020.12

第十四届 D2 前端技术论坛 · 微前端专场

演讲《标准微前端架构在蚂蚁的落地实践》 #有知

2020.4

qiankun 正式发布 2.0 版本

官宣文章 [《目标是最完善的微前端解决方案——qiankun2.0》](#)

2020.5

Github 累计 5.1k star，被许多团队选用

收获来自 single-spa 团队的肯定

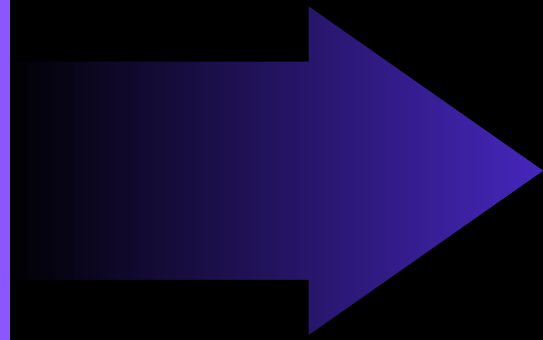
微前端的两种形态



子应用 还是 组件？

qiankun@2.x 定位转变

基于路由的
微前端框架



微应用加载器

从 单实例场景 跨越到 多实例场景

组件化加载能力

更复杂的应用编排

解锁微前端新用法

qiankun@2.x 加载微应用

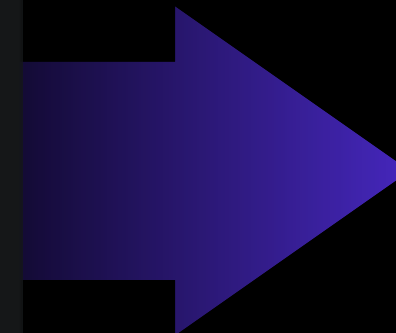
```
import { loadMicroApp } from 'qiankun';

/** 微应用的 React 封装 */
const MicroApp = () => {
  const containerRef = React.useRef()

  React.useEffect(() => {
    const microApp = loadMicroApp({
      name: "microApp",
      entry: "https://localhost:7001/micro-app.html",
      container: containerRef.current
    })

    return () => {
      microApp.mountPromise.then(() => microApp.unmount())
    }
  }, [])

  return <div ref={containerRef} />
}
```



```
<Route
  path="app1"
  component={
    <MicroApp />
  }
/>
```

路由级使用

```
<Container>
  <AppA />
  <MicroApp />
</Container>
```

组件级使用

微应用：混合研发



结合 NoCode/LowCode 与传统 ProCode 的 混合研发方案

欢迎加入

未来

qiankun@3.x 期待你的想法，欢迎你的共建

书籍推荐



Ruby 之父

深入理解程序设计背后的故事

Q&A

THANKS